Lab Report Submission

Course Code: SE215
Course Name: Algorithm Design and Analysis
Experiment No: Lab Report
Experiment Name: SE 215 LAB REPORT SUBMISSION

Submitted To

Name: Syed Shams Islam
Designation: Lecturer
Department: Software Engineering
Daffodil International University

Submitted By

Name: SUMIAYA HAQUE PRITHA
ID: 242-35-270
Section: 43 L2
Semester: 5th
Department: Software Engineering
Daffodil International University

Submission Date: 21/04/2026

Problem 1:

Code

```
#include <stdio.h>

int find_median(int *arr, int size) {
    int mid = size / 2;
   for (int i = 0; i <= mid; i++) {
     int smallest = i;
        for (int j = i + 1; j < size; j++) {
    if (arr[j] < arr[smallest]) {
       smallest = j;
           }
        }

        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;
    }

    return arr[mid];
}

int main() {
    int a1[] = {7, 2, 10, 9, 1};
    printf("%d\n", find_median(a1, 5));

    int a2[] = {4, 1, 8, 3};
    printf("%d\n", find_median(a2, 4));

return 0;
 }
```

Output:

7

3

Problem 2:

Code

```
#include <stdio.h>

int find_kth_largest(int *arr, int size, int k) {
        for (int i = 0; i < k; i++) {
            for (int j = 0; j < size - i - 1; j++) {
```

```
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
        return arr[size - k];
    }

    int main() {
        int a1[] = {3, 1, 7, 4, 9, 2};
        printf("%d\n", find_kth_largest(a1, 6, 2));

        int a2[] = {10, 5, 8, 3};
        printf("%d\n", find_kth_largest(a2, 4, 4));

        return 0;
    }
```

output:

7
3

Problem 3
code:

```
#include <stdio.h>

void merge_sorted(int *a, int n, int *b, int m, int *out) {
    int count = 0;

    for (int i = 0; i < n; i++) {
        int key = a[i];
        int j = count - 1;
        while (j >= 0 && out[j] > key) {
            out[j + 1] = out[j];
            j--;
        }
        out[j + 1] = key;
        count++;
    }

    for (int i = 0; i < m; i++) {
        int key = b[i];
```

```
        int j = count - 1;
        while (j >= 0 && out[j] > key) {
            out[j + 1] = out[j];
            j--;
        }
        out[j + 1] = key;
        count++;
    }
}

int main() {
    int a[] = {5, 1, 9}, n = 3;
    int b[] = {3, 7, 2, 6}, m = 4;
    int out[7];

    merge_sorted(a, n, b, m, out);

    for (int i = 0; i < 7; i++) {
        printf("%d ", out[i]);
    }
    printf("\n");

    return 0;
}
```

output:

1 2 3 5 6 7 9

Problem 4.1:

```
#include <stdio.h>

long long factorial(int n) {
    if (n == 0) return 1;

    long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main() {
    printf("%lld\n", factorial(0));
    printf("%lld\n", factorial(5));
    printf("%lld\n", factorial(10));

    return 0;
```

```
}
```

Output:

```
1
120
3628800
```

Problem 4.2:

```
#include <stdio.h>
long long fibonacci(int n) {
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
int main() {
    printf("fibonacci(0) => %lld\n", fibonacci(0));
    printf("fibonacci(7) => %lld\n", fibonacci(7));
    printf("fibonacci(10) => %lld\n", fibonacci(10));
    return 0;
}
```

Output :

```
fibonacci(0) => 0
fibonacci(7) => 13
fibonacci(10) => 55
```

Problem 4.3:

Code:

```
#include <stdio.h>
int count_digits(int n) {
    if (n == 0) {
        return 1;
    }
    int count = 0;
    while (n > 0) {
        n = n / 10;
        count++;
    }
    return count;
}
int main() {
```

```
    printf("count_digits(0) => %d\n", count_digits(0));
    printf("count_digits(9) => %d\n", count_digits(9));
    printf("count_digits(100) => %d\n", count_digits(100));
    printf("count_digits(98765) => %d\n", count_digits(98765));
    return 0;
}
```

Output :

```
count_digits(0) => 1
count_digits(9) => 1
count_digits(100) => 3
count_digits(98765) => 5
```

Problem 4.4:

code :

```
#include <stdio.h>
double power(double base, int exp) {
    double result = 1.0;
    for (int i = 0; i < exp; i++) {
        result *= base;
    }
    return result;
}
int main() {
      printf("power(2.0, 0) => %.1f\n", power(2.0, 0));              printf("power(3.0, 4) => %.1f\n",
power(3.0, 4));
    printf("power(2.5, 3) => %.3f\n", power(2.5, 3));
    return 0;
}
```

Output :

```
power(2.0, 0) => 1.0
power(3.0, 4) => 81.0
power(2.5, 3) => 15.625
```

Problem 4.5:

Code:

```
#include <stdio.h>
int is_palindrome(char *str, int left, int right) {
    if (left >= right) {
        return 1;
    }
    if (str[left] != str[right]) {
        return 0;
```

```
    }
    return is_palindrome(str, left + 1, right - 1);
}
int main() {
    printf("is_palindrome(\"racecar\", 0, 6) => %d\n", is_palindrome("racecar", 0, 6));
    printf("is_palindrome(\"hello\", 0, 4) => %d\n", is_palindrome("hello", 0, 4));
    printf("is_palindrome(\"a\", 0, 0) => %d\n", is_palindrome("a", 0, 0));
    printf("is_palindrome(\"abba\", 0, 3) => %d\n", is_palindrome("abba", 0, 3));
    return 0;
}
```

Output:

```
is_palindrome("racecar", 0, 6) => 1
is_palindrome("hello", 0, 4) => 0
is_palindrome("a", 0, 0) => 1
is_palindrome("abba", 0, 3) => 1
```

Problem 4.6:

```
#include <stdio.h>
int array_sum(int *arr, int size) {
    if (size <= 0) {
        return 0;
    }
    return arr[size - 1] + array_sum(arr, size - 1);
}
int main() {
    int arr1[] = {1, 2, 3, 4, 5};
    printf("array_sum({1,2,3,4,5}, 5) => %d\n", array_sum(arr1, 5));
    int arr2[] = {-3, 7, 0};
    printf("array_sum({-3, 7, 0}, 3) => %d\n", array_sum(arr2, 3));
    int arr3[] = {42};
    printf("array_sum({42}, 1) => %d\n", array_sum(arr3, 1));
    return 0;
}
```

Output :

```
array_sum({1,2,3,4,5}, 5) => 15
array_sum({-3, 7, 0}, 3) => 4
array_sum({42}, 1) => 42
```

Problem 4.7:

Code:

```
#include <stdio.h>
int array_max(int *arr, int size) {
    if (size == 1) {
```

```
        return arr[0];
    }
    int max_of_rest = array_max(arr, size - 1);
    if (arr[size - 1] > max_of_rest) {
        return arr[size - 1];
    } else {
        return max_of_rest;
    }
}
int main() {
    int arr1[] = {3, 1, 9, 2, 7};
    printf("array_max({3, 1, 9, 2, 7}, 5) => %d\n", array_max(arr1, 5));
    int arr2[] = {-5, -1, -8};
    printf("array_max({-5, -1, -8}, 3) => %d\n", array_max(arr2, 3));
    int arr3[] = {42};
    printf("array_max({42}, 1) => %d\n", array_max(arr3, 1));
    return 0;
}
```

Output :

```
array_max({3, 1, 9, 2, 7}, 5) => 9
array_max({-5, -1, -8}, 3) => -1
array_max({42}, 1) => 42
```

Problem 4.8:

```
#include <stdio.h>
int binary_search(int *arr, int left, int right, int target) {
    if (left > right) {
        return -1;
    }
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) {
        return mid;
    }
    if (target < arr[mid]) {
        return binary_search(arr, left, mid - 1, target);
    }
    return binary_search(arr, mid + 1, right, target);
}
int main() {
    int arr1[] = {1, 3, 5, 7, 9};
    printf("binary_search({1,3,5,7,9}, 0, 4, 7) => %d\n", binary_search(arr1, 0, 4, 7));
    printf("binary_search({1,3,5,7,9}, 0, 4, 4) => %d\n", binary_search(arr1, 0, 4, 4));
    int arr2[] = {2};
    printf("binary_search({2}, 0, 0, 2) => %d\n", binary_search(arr2, 0, 0, 2));
    return 0;
}
```

Output :

binary_search({1,3,5,7,9}, 0, 4, 7) => 3
binary_search({1,3,5,7,9}, 0, 4, 4) => -1
binary_search({2}, 0, 0, 2) => 0

Problem 5:

Code:

```
#include <stdlib.h>
void merge(int *arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
    free(L);
    free(R);
}
void merge_sort(int *arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort(arr, left, mid);
        merge_sort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

```
void print_array(int *arr, int size) {
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int size = 7;
    printf("Original: ");
    print_array(arr, size);
    merge_sort(arr, 0, size - 1);
    printf("Sorted:   ");
    print_array(arr, size);

    return 0;
}
```

Output :

```
Original: 38 27 43 3 9 82 10
Sorted:   3 9 10 27 38 43 82
```

Problem 6:

code:

```
#include <stdio.h>
int swap_count = 0;
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
    swap_count++;
}
int partition(int *arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quick_sort(int *arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quick_sort(arr, low, pi - 1);
```

```
        quick_sort(arr, pi + 1, high);
    }
}
void print_array(int *arr, int size) {
    for (int i = 0; i < size; i++) printf("%d ", arr[i]);
    printf("\n");
}
int main() {

    int arr1[] = {10, 7, 8, 9, 1, 5};
    int n1 = 6;
    swap_count = 0;
    quick_sort(arr1, 0, n1 - 1);
    printf("Example Output: ");
    print_array(arr1, n1);
    printf("Total Swaps: %d\n\n", swap_count);

    int arr2[] = {5, 4, 3, 2, 1};
    int n2 = 5;
    swap_count = 0;
    quick_sort(arr2, 0, n2 - 1);
    printf("Reverse Sorted Case: ");
    print_array(arr2, n2);
    printf("Total Swaps: %d\n", swap_count);
    return 0;
}
```

Output :

```
Example Output: 1 5 7 8 9 10
Total Swaps: 5
Reverse Sorted Case: 1 2 3 4 5
Total Swaps: 8
```

Problem 7:

code

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int weight;
    int value;
} Item;
int compare(const void *a, const void *b) {
    Item *item1 = (Item *)a;
    Item *item2 = (Item *)b;
```

```
    double ratio1 = (double)item1->value / item1->weight;
    double ratio2 = (double)item2->value / item2->weight;

    if (ratio1 < ratio2) return 1;
    if (ratio1 > ratio2) return -1;
    return 0;
}
double fractional_knapsack(Item *items, int n, int capacity) {
    qsort(items, n, sizeof(Item), compare);
    double total_value = 0.0;
    int current_weight = 0;

    for (int i = 0; i < n; i++) {
        if (current_weight + items[i].weight <= capacity) {
            current_weight += items[i].weight;
            total_value += items[i].value;
        }

        else {
            int remaining_capacity = capacity - current_weight;
            total_value += items[i].value * ((double)remaining_capacity / items[i].weight);
            break;
        }
    }
    return total_value;
}
int main() {

    Item items1[] = {{10, 60}, {20, 100}, {30, 120}};
    int n1 = 3;
    int capacity1 = 50;
    printf("Example 1 Output: %.2f\n", fractional_knapsack(items1, n1, capacity1));
    Item items2[] = {{5, 50}, {10, 60}, {15, 90}};
    int n2 = 3;
    int capacity2 = 20;
    printf("Example 2 Output: %.2f\n", fractional_knapsack(items2, n2, capacity2));

    return 0;
}
```

Output :
Example 1 Output: 240.00
Example 2 Output: 140.00

Problem 8a:

code:

```
#include <stdio.h>
#include <limits.h>
```

```c
#include <stdbool.h>

#define V 5

int min_key(int key[], bool mstSet[]) {
    int min = INT_MAX, min_index = -1;

    for (int v = 0; v < V; v++)
        if (mstSet[v] == false && key[v] < min)
            min = key[v], min_index = v;

    return min_index;
}
void prims_mst(int graph[V][V]) {
    int parent[V], key[V];
    bool mstSet[V];

    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < V - 1; count++) {
        int u = min_key(key, mstSet);
        mstSet[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    int total = 0;
    printf("Edge Weight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d %d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
    printf("Total weight: %d\n", total);
}
int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
```

```
        {0, 5, 7, 9, 0}
    };

    prims_mst(graph);

    return 0;
}
```

Output :

```
Edge Weight
0 - 1 2
1 - 2 3
0 - 3 6
1 - 4 5
Total weight: 16
```

Problem 8b:

code:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int src, dest, weight;
} Edge;

typedef struct {
    int parent, rank;
} Subset;
int find(Subset subsets[], int i) {
    if (subsets[i].parent != i)
        subsets[i].parent = find(subsets, subsets[i].parent);
    return subsets[i].parent;
}
void Union(Subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank)
        subsets[xroot].parent = yroot;
    else if (subsets[xroot].rank > subsets[yroot].rank)
        subsets[yroot].parent = xroot;
    else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}
int compareEdges(const void* a, const void* b) {
```

```
    return ((Edge*)a)->weight - ((Edge*)b)->weight;
}

void kruskals_mst(Edge edges[], int E, int V) {
    qsort(edges, E, sizeof(Edge), compareEdges);

    Subset subsets[V];
    for (int i = 0; i < V; i++) {
        subsets[i].parent = i;
        subsets[i].rank = 0;
    }

    int total = 0;
    int count = 0;
    int i = 0;

    printf("Edge Weight\n");

    while (count < V - 1) {
        Edge next = edges[i++];

        int x = find(subsets, next.src);
        int y = find(subsets, next.dest);

        if (x != y) {
            printf("%d - %d %d\n", next.src, next.dest, next.weight);
            total += next.weight;
            Union(subsets, x, y);
            count++;
        }
    }

    printf("Total weight: %d\n", total);
}
int main() {
    int V = 5;
    int E = 7;
    Edge edges[] = {
        {0,1,2},{0,3,6},{1,2,3},{1,3,8},
        {1,4,5},{2,4,7},{3,4,9}
    };

    kruskals_mst(edges, E, V);

    return 0;
}
```

Output :

```
Edge Weight
0 - 1 2
```

1 - 2 3
1 - 4 5
0 - 3 6
Total weight: 16

Problem 9a:

Code:

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 5

int min_distance(int dist[], bool visited[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++)
        if (visited[v] == false && dist[v] <= min)
            min = dist[v], min_index = v;

    return min_index;
}

void dijkstra(int graph[V][V], int src) {
    int dist[V];
    bool visited[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = false;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = min_distance(dist, visited);
        visited[u] = true;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}
```

```
int main() {
    int graph[V][V] = {
        { 0, 4, 0, 0, 8 },
        { 4, 0, 8, 0, 11},
        { 0, 8, 0, 7, 0 },
        { 0, 0, 7, 0, 9 },
        { 8, 11, 0, 9, 0 }
    };
    dijkstra(graph, 0);

    return 0;
}
```

Output :

```
Vertex  Distance from Source
0       0
1       4
2       12
3       17
4       8
```

Problem 9b:

Code:

```
#include <stdio.h>
#include <limits.h>

#define INF (INT_MAX / 2)

typedef struct {
    int src, dest, weight;
} Edge;

void bellman_ford(int V, int E, Edge *edges, int src) {
    int dist[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INF;
    }
    dist[src] = 0;
    for (int i = 1; i <= V - 1; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j].src;
            int v = edges[j].dest;
            int weight = edges[j].weight;
            if (dist[u] != INF && dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
            }
        }
    }
```

```
    for (int j = 0; j < E; j++) {
        int u = edges[j].src;
        int v = edges[j].dest;
        int weight = edges[j].weight;
        if (dist[u] != INF && dist[u] + weight < dist[v]) {
            printf("Graph contains a negative-weight cycle\n");
            return;
        }
    }
    printf("Vertex   Distance from Source\n");
    for (int i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("%d \t INF\n", i);
        else
            printf("%d \t %d\n", i, dist[i]);
    }
}

int main() {
    int v_count = 5;
    int e_count = 10;
    Edge edges[] = {
        {0, 1, 6}, {0, 2, 7}, {1, 2, 8}, {1, 3, 5}, {1, 4, -4},
        {2, 3, -3}, {2, 4, 9}, {3, 1, -2}, {4, 0, 2}, {4, 3, 7}
    };

    bellman_ford(v_count, e_count, edges, 0);

    return 0;
}
```

Output :

```
Vertex   Distance from Source
0        0
1        2
2        7
3        4
4        -2
```

Problem 10a:

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack_01(int *weights, int *values, int n, int W) {

    int **dp = (int **)malloc((n + 1) * sizeof(int *));
    for (int i = 0; i <= n; i++) {
        dp[i] = (int *)malloc((W + 1) * sizeof(int));
    }
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            else if (weights[i - 1] <= w) {
                dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]],
                               dp[i - 1][w]);
            }
            else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    int result = dp[n][W];
    for (int i = 0; i <= n; i++) {
        free(dp[i]);
    }
    free(dp);

    return result;
}

int main() {
    int w1[] = {2, 3, 4, 5};
    int v1[] = {3, 4, 5, 6};
    printf("Example 1 Output: %d\n", knapsack_01(w1, v1, 4, 5));
    int w2[] = {1, 3, 4, 5};
    int v2[] = {1, 4, 5, 7};
    printf("Example 2 Output: %d\n", knapsack_01(w2, v2, 4, 7));

    return 0;
}
```

Output :

Example 1 Output: 7
Example 2 Output: 9

Problem 10b:
code:

```
#include <stdio.h>
#include <limits.h>
#define INF (INT_MAX - 1)
int min_coins(int *coins, int n, int amount) {
    int dp[amount + 1];
    dp[0] = 0;
    for (int i = 1; i <= amount; i++) {
        dp[i] = INF;
    }

    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i) {
                int res = dp[i - coins[j]];

                if (res != INF && res + 1 < dp[i]) {
                    dp[i] = res + 1;
                }
            }
        }
    }

    return (dp[amount] == INF) ? -1 : dp[amount];
}

int main() {
    int c1[] = {1, 5, 6, 9};
    printf("Example 1 Output: %d\n", min_coins(c1, 4, 11));

    int c2[] = {2};
    printf("Example 2 Output: %d\n", min_coins(c2, 1, 3));

    int c3[] = {1, 2, 5};
```

```
    printf("Example 3 Output: %d\n", min_coins(c3, 3, 11));

    return 0;
}
```

Output :

Example 1 Output: 2
Example 2 Output: -1
Example 3 Output: 3

Problem 10c:
Code:

```
#include <stdio.h>
int count_ways(int *coins, int n, int amount) {
    int dp[amount + 1];
    for (int i = 0; i <= amount; i++) {
        dp[i] = 0;
    }
    dp[0] = 1;
    for (int i = 0; i < n; i++) {
        int current_coin = coins[i];
        for (int a = current_coin; a <= amount; a++) {
            dp[a] += dp[a - current_coin];
        }
    }
    return dp[amount];
}

int main() {
    int c1[] = {1, 2, 3};
    printf("Example 1 Output: %d\n", count_ways(c1, 3, 4));
    int c2[] = {2, 5, 3, 6};
    printf("Example 2 Output: %d\n", count_ways(c2, 4, 10));
    return 0;
}
```

Output :

Example 1 Output: 4
Example 2 Output: 5

Problem 10d:

Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int lcs(char *s1, char *s2, int m, int n) {
    int dp[m + 1][n + 1];
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    int index = dp[m][n];
    char *lcs_str = (char *)malloc((index + 1) * sizeof(char));
    lcs_str[index] = '\0';

    int i = m, j = n;
    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) {
            lcs_str[index - 1] = s1[i - 1];
            i--; j--; index--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }
    printf("Actual LCS: %s\n", lcs_str);
    int result = dp[m][n];
    free(lcs_str);
    return result;
}
int main() {
    char s1[] = "ABCBDAB";
```

```
    char s2[] = "BDCAB";
    printf("LCS Length: %d\n\n", lcs(s1, s2, strlen(s1), strlen(s2)));

    char s3[] = "AGGTAB";
    char s4[] = "GXTXAYB";
    printf("LCS Length: %d\n\n", lcs(s3, s4, strlen(s3), strlen(s4)));
    return 0;
}
```

Output :

```
Actual LCS: BDAB
LCS Length: 4

Actual LCS: GTAB
LCS Length: 4
```

Problem 10e:

Code:

```
#include <stdio.h>
int lis(int *arr, int n) {
    if (n == 0) return 0;
    int dp[n];
    int max_lis = 1;
    for (int i = 0; i < n; i++) {
        dp[i] = 1;
    }
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (arr[i] > arr[j] && dp[i] < dp[j] + 1) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > max_lis) {
            max_lis = dp[i];
        }
    }
    return max_lis;
}
int main() {
    int arr1[] = {10, 9, 2, 5, 3, 7, 101, 18};
    printf("LIS Length: %d\n", lis(arr1, 8));
```

```
    int arr2[] = {0, 1, 0, 3, 2, 3};
    printf("LIS Length: %d\n", lis(arr2, 6));
    int arr3[] = {7, 7, 7, 7};
    printf("LIS Length: %d\n", lis(arr3, 4));
    return 0;
}
```

Output :

```
LIS Length: 4
LIS Length: 4
LIS Length: 1
```